

PART 2

**SENIOR
LEVEL
QUES-
TIONS**

CHAPTER 8

BACKGROUND JOBS

64. How can we perform long-running operations in background?

Android provides a few ways of this depending on our needs. To name a few of them: **ThreadPool**, **Foreground Services** and **WorkManager**.

Let's explain them one by one.

ThreadPools:

For work that should only be done when our app is in the foreground, we use **ThreadPools**. **ThreadPools** provide a group of background threads that accept and enqueue submitted work. If we need to monitor system triggers during this time, we use dynamically-registered broadcast receivers to monitor OS state and triggers.

Foreground Services:

For work that must execute to completion, if we need the work to execute immediately, we use a **foreground service**. Using a foreground service tells the system that the app is doing something

important and they shouldn't be killed. Fore-ground services are visible to users via a non-dismissible notification in the notification tray.

WorkManager:

For work that must execute to completion and is deferrable, we use **WorkManager**. **WorkManager** is an Android library that gracefully runs deferrable background work when the work's triggers (like appropriate network state and battery conditions) are satisfied. **WorkManager** uses the framework **JobScheduler** whenever possible, to help optimize battery life and batch jobs. On devices below Android 6.0 (API level 23), **WorkManager** attempts to use *Firebase Job-Dispatcher* if it's already an included dependency of our app; otherwise, **WorkManager** falls back to a custom *AlarmManager* implementation to gracefully handle our background work.

65. What happens if Activity dies while AsyncTask is running?

Lifecycle of an Activity and that of an AsyncTask are separated. That means if Activity dies while AsyncTask is still running, the AsyncTask won't stop. It will keep working. But in `onPostExecute()` method if **AsyncTask** tries to update UI, we get into trouble.

Since Activity is already dead, system throws an exception:

```
java.lang. IllegalArgumentException : View not  
attached to window manager
```

To get rid of this, we should always check if Activity is alive before updating UI in `onPostExecute()` method of **AsyncTask**.

```
protected fun onPostExecute(file_url: String) {  
    if ( this @YourActivity .isDestroyed()) {  
        // or call isFinishing() if min sdk version < 17  
        return  
    }  
    dismissProgressDialog()  
    something(note)  
}
```

66. How can we benefit from Handlers?

Handlers are objects for managing threads. It receives messages and writes code on how to handle the message. They run outside of the activity's lifecycle, so they need to be cleaned up properly or else you will have thread leaks.

```
Handler(Looper.getMainLooper()).post( Runnable  
{  
    kotlin. run {  
        //This runs in the main thread  
    }  
})
```

Handlers allow communicating between the background thread and the main thread.

```
val handler = object : Handler() {  
    override fun handleMessage(msg: Message) {  
        // process incoming messages here  
    }  
}
```



```

        // this will run in the thread, which instantiates
        it
    }
}
// Let's send a message to Handler
val msg = Message()
msg.obj = "Ali send message"
handler.sendMessage(msg)

```

A Handler class is preferred when we need to perform a background task repeatedly after every x seconds/minutes or to be executed after a delay..

```

Handler().postDelayed(
    Runnable { kotlin.run {
        Log.d( TAG , "This will run after 2
seconds." ) }
    },
    2000 )

```

67. What is difference between Service, Intent Service, AsyncTask & Threads?

First of all these are the classes that we use to perform tasks on background. In real world applications there are some really complicated scenarios that we need to take care of. We need to think judiciously to select which structure to use.

To explain them briefly;

Android service is a component that is used to perform operations on the background such as

playing music. It doesn't have any UI (user interface). The service runs in the background but be aware that it runs on the main thread. Android Oreo has restricted services while app is not on foreground but there are a few solutions for this.

AsyncTask allows us to perform asynchronous work on our user interface. It performs the blocking operations in a worker thread and then publishes the results on the UI thread, without requiring you to handle threads and/or handlers yourself. We should deny it for long running operations.

IntentService is a base class for Services that handle asynchronous requests (expressed as Intents) on demand. Clients send requests through `startService(Intent)` calls; the service is started as needed, handles each Intent in turn using a worker thread, and stops itself when it runs out of work.

A **thread** is a single sequential flow of control within a program. Threads can be thought of as mini-processes running within a main process. This is the base of all background jobs and it's specific to Java. So, we should deny using it unless it's the last option for us.

68. What are disadvantages of AsyncTask?

AsyncTask is flexible and easy to use. But in some cases there are some problems with it. We should avoid using AsyncTask for long running operations. Some problems in AsyncTask are;

When you rotate your screen, Activity gets destroyed, so AsyncTask will not have a valid reference to publish data from `onPostExecute()`. In order to retain it, you need to `usesetRetainState(true)` if calling from fragment or `onConfigChanges()` if calling from activity method of an activity.

When the Activity is restarted, your AsyncTask's reference to the Activity is invalid, so `onPostExecute()` will have no effect on the new Activity.

If activity gets finished, AsyncTask execution will not be cancelled automatically, you need to cancel them else they will keep on running in the background.

If any exception occurs while performing network task, you need to handle them manually.

69. How can you manage multiple Threads running at the same time?

To allow multiple tasks to run at the same time, we need to provide a managed collection of threads. To do this, we use an instance of **ThreadPoolExecutor**, which runs a task from a queue when a thread in its pool becomes free. To run a task, all we have to do is add it to the queue.

A thread pool can run multiple parallel instances of a task, so we should ensure that our code is thread-safe. Enclose variables that can be accessed by more than one thread in a `synchro-`

nized block. This approach will prevent one thread from reading the variable while another is writing to it. Typically, this situation arises with static variables, but it also occurs in any object that is only instantiated once.

For instance, let's assume you want to synchronise threads for downloading photos. You need to create an instance of `ThreadPoolExecutor` class.

```
object PhotoManager {  
    ...  
    // Sets the amount of time an idle thread waits  
    before terminating  
    private const val KEEP_ALIVE_TIME = 1L  
    // Sets the Time Unit to seconds  
    private val KEEP_ALIVE_TIME_UNIT = Time-  
Unit.SECONDS  
    // Creates a thread pool manager  
    private val mDecodeThreadPool : Thread-  
PoolExecutor = ThreadPoolExecutor(  
        NUMBER_OF_CORES,    // Initial pool size  
        NUMBER_OF_CORES,    // Max pool size  
        KEEP_ALIVE_TIME ,  
        KEEP_ALIVE_TIME_UNIT ,  
        mDecodeWorkQueue  
    )  
}
```

70. What are advantages of AsyncTask over Thread?

Thread should be used to separate long running operations from main thread so that performance is improved. But it can't be cancelled elegantly and it can't handle configuration changes of Android. We can't update UI from Thread.

```

val thread = Thread( Runnable {
    val bitmap = processBitMap( "image.png" );
    mImageView.post( Runnable () {
        mImageView.setImageBitmap(bitmap);
    } )
} ).start()

```

AsyncTask can be used to handle work items shorter than 5ms in duration. With AsyncTask, we can update UI unlike Java *Thread* . But many long running tasks will choke the performance.

```

class DownloadFilesTask: AsyncTask<URL, Integer, Long>() {
    override fun onPreExecute() {
        super .onPreExecute()
    }

    override fun doInBackground( vararg url: URL?): Long {
        return 0
    }

    override fun onProgressUpdate( vararg values: Integer?) {
        super .onProgressUpdate(*values)
    }

    override fun onPostExecute(result: Long?) {
        super .onPostExecute(result)
    }
}

// How to call
DownloadFilesTask().execute(url1, url2, url3);

```

The three types used by an asynchronous task are the following:

Params, the type of the parameters sent to the task upon execution.

Progress, the type of the progress units published during the background computation.

Result, the type of the result of the background computation.

71. What is onTrimMemory() method? When does it called?

onTrimMemory(): Called when the operating system has determined that it is a good time for a process to trim unneeded memory from its process. This will happen for example when it goes in the background and there is not enough memory to keep as many background processes running as desired.

Android can reclaim memory for from our app in several ways or kill our app entirely if necessary to free up memory for critical tasks. To help balance the system memory and avoid the system's need to kill our app process, we can implement the **ComponentCallbacks2** interface in our Activity classes. The provided **onTrimMemory()** callback method allows our app to listen for memory related events when our app is in either the foreground or the background, and then release objects in response to app lifecycle or system events that indicate the system needs to reclaim memory.

For instance;

```
public class MainActivity : AppCompatActivity(),
ComponentCallbacks2 {
    override fun onTrimMemory(level: Int) {
        super.onTrimMemory(level)
        // Determine which lifecycle or system event was
        raised.
        when (level) {
            ComponentCallbacks2.TRIM_MEMORY_UI_HIDDEN -> {
```

```

        // Release any UI objects that currently hold
        memory.
    }
    ComponentCallbacks2.TRIM_MEMORY_RUNNING_MODERATE,
    ComponentCallbacks2.TRIM_MEMORY_RUNNING_LOW,
    ComponentCallbacks2.TRIM_MEMORY_RUNNING_CRITICAL -> {
        // Release any memory that your app doesn't
        need to run.
    }
    ComponentCallbacks2.TRIM_MEMORY_BACKGROUND,
    ComponentCallbacks2.TRIM_MEMORY_MODERATE,
    ComponentCallbacks2.TRIM_MEMORY_COMPLETE -> {
        // Release as much memory as the process
        can.
    }
    else -> { // Release any non-critical data
        structures.}
    }
}
}
}

```

72. How do you find memory leaks in an Android application?

First of all, in Android Studio we have Memory Profiler tool. It helps us identify memory leaks and memory churn that can lead to stutter, freezes, and even app crashes. It shows a real time graph of our app's memory use, lets us capture a heap dump, force garbage collections, and track memory allocations.

Besides that we have the famous LeakCanary library. It is really easy to use and it again helps us to find memory leaks.

But while developing apps our priority should be to avoid things that cause memory leaks more than trying to find leaks after development. So, the question should be “How to avoid memory leaks?”.

Ok?

We have a bunch of things to avoid memory leaks in our apps. Let me roughly mention a few of them.

Never hold the reference of ui specific object in a background task, as it leads to memory leak.

Do not use static views as it is always available, static views never get killed.

Never use context as static. Context is an expensive object.

Use application context if possible and use activity context only if required.

```
class MainActivity : AppCompatActivity() {  
    companion object {  
        private val button : Button? = null //  
        NEVER USE LIKE THIS  
        private val context : Context? = null //  
        NEVER USE LIKE THIS  
    }  
}
```

Do not forget to unregister your listeners in `onPause()` / `onStop()` / `onDestroy()` method. By not unregistering, it always keeps the activity alive and keeps waiting for listeners.

If you are using an inner class, use this as static because static class does not need the outer class implicit reference. Using inner

class as non static makes the outer class alive so it is better to avoid.

And if you are using views in static class, pass it in the constructor and use it as a weak reference.

Avoid putting views in collections that do not have clear memory pattern.

```
class LocationListenerActivity : Activity(), LocationListener {  
    public override fun onStart() {  
        LocationListener.get().register( this )  
    }  
  
    override fun onLocationChanged(location: Location?) {  
    }  
    public override fun onStop() {  
        LocationListener.get().unregister( this )  
    }  
}
```

73. What are ways of scheduling jobs intelligently?

Android SDK provides several facilities to help us schedule work intelligently. First of the is **JobScheduler** .

JobScheduler:

JobScheduler does scheduling work for us that it improves app's performance along with aspects of the system health such as battery life.

JobScheduler is implemented in the platform, which allows it to collect information about jobs that need to run across all apps. This informa-

tion is used to schedule jobs to run at, or around, the same time.

The `JobScheduler` API allows us to specify robust conditions for executing tasks, along with centralized task scheduling across the device for optimal system health. It is also suitable for small tasks like clearing a cache, and for large ones such as syncing a database to the cloud.

AlarmManager API:

The **AlarmManager API** is another option that the framework provides for scheduling tasks. This API is useful in cases in which an app needs to post a notification or set off an alarm at a very specific time.

You should only use this API for tasks that must execute at a specific time, but don't require the other, more robust, execution conditions that `JobScheduler` allows you to specify, such as device idle and charging detect.

Firebase JobDispatcher:

Firebase `JobDispatcher` is an open-source library that provides an API similar to `JobScheduler` in the Android platform. Firebase `JobDispatcher` serves as a `JobScheduler`-compatibility layer for apps targeting versions of Android lower than 5.0 (API level 21).

74. What is ANR? When it happens? How to avoid it?

First of all ANR means Application Not Responding. It happens when application is blocked, un-

able to work etc... The operating system interrupts and says "This is an ANR" and shuts the app down.

According to the official documentation of Android, there are two scenarios for an ANR to happen.

First , if there is no response for an input event for 5 seconds, then this is an ANR. For instance you have pressed a key on keyboard and for five seconds there isn't any response, Android OS interrupts and throws an ANR.

Secondly , assume that you have a BroadcastReceiver. By the time it receives a broadcast message, then it has 10 seconds to finish its execution. Otherwise it is yet another ANR.

75. Have a look at the below code. Does it cause ANR?

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState:  
Bundle?, savedInstanceState: PersistableBundle?) {  
        super.onCreate(savedInstanceState,  
persistentState)  
  
        Thread.sleep( 15000 ) // sleep for 15 seconds  
    }  
}
```

Ok, this a very tricky question. Let's have a look to the code. There seems an Activity and in `onCreate()` method there is a `Thread.sleep( 15000 )` code that makes thread sleep for 15 seconds.

Does it cause ANR?

First of all, we need to know the definition of ANR. Google's official documentation says that if there is no response for an input event for 5 seconds, then this is an ANR. But the tricky part here is it must be an input event, such as pressing a key on keyboard.

Since this code makes thread sleep more than five second, it is not an input event. So, the answer is NO, it doesn't cause an ANR.

CHAPTER 9

RECYCLERVIEW, LOADING DATA WITH PERFORMANCE

76. How does RecyclerView differ from Listview?

Recyclerview implements the **ViewHolder** pattern whereas it is not mandatory in a ListView.

A RecyclerView recycles and reuses cells when scrolling .

What is a ViewHolder Pattern?

A ViewHolder object stores each of the component views inside the tag field of the Layout, so we can immediately access them without the need to look them up repeatedly.

In ListView, the code might call `findViewById()` frequently during the scrolling of ListView, which can slow down performance. Even when the Adapter returns an inflated view for recycling, we still need to look up the elements and update them. A way around repeated use of `findViewById()` is to use the **ViewHolder** design pattern.

In a ListView, the only type of view available is the vertical ListView. A RecyclerView decouples list from its container so we can put list items easily at run time in the different containers (LinearLayout, GridLayout) by setting **LayoutManager** .

Lists are lacking in support of good animations, but the RecyclerView brings a whole new dimension to it. RecyclerViews use **Item Animators** for supporting animations.

To sum up, ViewHolder pattern, LayoutManager and Item Animators are the basic advantages of RecyclerView over ListView.

77. What is SnapHelper? Can you briefly explain how to use it?

SnapHelper is a helper class that helps in snapping any child view of the RecyclerView. For

example, you can snap the `firstVisibleItem` of the `RecyclerView` as you must have seen in the play store application that the `firstVisibleItem` will be always completely visible when scrolling comes to the idle position

```
val snapHelper = LinearSnapHelper()
snapHelper.attachToRecyclerView(yourRecyclerView)
```

78. How can we efficiently update `RecyclerView` when only a small amount of data is updated?

When adapter's data is changed we have to notify adapter in order to update `RecyclerView`. The most common way of this is calling `notifyDataSetChanged()` method.

But the problem is this method causes adapter to update all the indices in data. That is going to slow down our app. If we are writing an adapter it will always be more efficient to use the more specific change events if we can. We should rely on `notifyDataSetChanged()` as a last resort.

There are overloaded versions of this methods:

```
notifyItemChanged(int position)
notifyItemInserted(int position)
notifyItemRemoved(int position)
notifyItemRangeChanged(int positionStart, int itemCount)
notifyItemRangeInserted(int positionStart, int itemCount)
notifyItemRangeRemoved(int positionStart, int itemCount)
```

The most appropriate method is `notifyItemRangeChanged (int positionStart, int itemCount)`.

This is an item change event, not a structural change event. It indicates that any reflection of the data in the given position range is out of date and should be updated. The items in the given range retain the same identity.

```
void notifyItemRangeChanged (int positionStart,  
int itemCount)
```

79. How does RecyclerView work?

RecyclerView is designed to display long lists of items. Say we want to display 100 row of items. A simple approach would be to just create 100 views, one for each row and lay all of them out.

But that would be wasteful because at any point of time, only 10 or so items could fit on screen and the remaining items would be off screen.

So RecyclerView instead creates only the 10 or so views that are on screen. This way you get 10x better speed and memory usage.

What if happens when you start scrolling and need to start showing next views?

Again a simple approach would be to create a new view for each new row that you need to show.

But this way by the time you reach the end of the list you will have created 100 views and your memory usage would be the same as in the first approach. And creating views takes time,

so your scrolling most probably wouldn't be smooth.

This is why RecyclerView takes advantage of the fact that as you scroll, new rows come on screen also old rows disappear off screen.

Instead of creating new view for each new row, an old view is recycled and reused by binding new data to it.

This happens inside the `onBindViewHolder()` method. Initially you will get new unused view holders and you have to fill them with data you want to display. But as you scroll you will start getting view holders that were used for rows that went off screen and you have to replace old data that they held with new data.

CHAPTER 10

DATABASE

80. Why do we use SQLite instead of SQL in mobile apps?

SQLite is a software library that implements a self-contained, serverless, zero-configuration, transactional SQL database engine.

Advantage of SQLite:

An SQLite database can also be queried and the data retrieval is much more robust.

The `android.database` and `android.database.sqlite` packages offer a higher-performance alternative where source compatibility is not an issue.

Android-databases created in Android are visible only to the application that created them

There is no file parsing and generating code to write and debug.

Content can be accessed and updated using powerful SQL queries, greatly reducing the complexity of the application code.

Extending the file format for new capabilities in later releases is as simple as adding new tables or new columns to existing tables.

The application file is portable across all operating systems, 32-bit and 64-bit and big- and little-endian architectures.

The application only has to load as much data as it needs, rather than reading the entire application file and holding a complete parse in memory. Startup time and memory consumption are reduced.

Small edits only overwrite the parts of the file that change, not the entire file, thus improving performance and reducing wear on SSD drives.

Multiple processes can attach to the same application file and can read and write without interfering with each other.

In many common cases, loading content from an SQLite database is faster than loading content out of individual files. See Internal Versus External BLOBs for additional information.

81. What are the SQLite storage classes?

Each value stored in an SQLite database (or manipulated by the database engine) has one of the following storage classes:

NULL . The value is a NULL value.

INTEGER . The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.

REAL . The value is a floating point value, stored as an 8-byte IEEE floating point number.

TEXT . The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE).

BLOB . The value is a blob of data, stored exactly as it was input.

82. What is DAO Pattern? How do we use it and what is its advantage?

Data Access Object Pattern or **DAO** pattern is used to separate low level data accessing API or operations from high level business services. Following are the participants in Data Access Object Pattern.

Data Access Object Interface - This interface defines the standard operations to be performed on a model object(s).

```
interface Dao {  
    fun getAllStudents(): List<Student>;  
}
```

Data Access Object concrete class - This class implements DAO interface. This class is responsible to get data from a data source which can be database / xml or any other storage mechanism.

```
class DummyDaoImpl : Dao {  
    override fun getAllStudents(): List<Student> {  
    }  
}  
class NetworkDaoImpl : Dao {  
    override fun getAllStudents(): List<Student> {  
    }  
}
```

Model Object or Value Object - This object is simple POJO containing get/set methods to store data retrieved using DAO class

```
data class Student( val name : String)
```

Main advantage of DAO pattern is that we can quickly change source of data in our app. While developing apps, we may want use dummy data before using real data. DAO pattern gives us this advantage.

ROOM persistence library also uses this pattern for a flexible structure.

83. How can you separate your data layer from presentation?

Android's framework team recently has introduced a new architecture. They want us to separate presentation layer and data layer. For this purpose they recommend us to use Repository Pattern.

Repository is a pattern and it is the single source of truth for all the app data. It has a clean API for UI to communicate with.

Whenever we need data, we go to Repository. It doesn't matter where the data comes from. From local db or network or file storage or SharedPreferences or somewhere else.

Here we have a simple Repository example:

```
class UserRepository( val userApi : UserApi, val
userDao : UserDao) {
    fun getUsers(): Observable<List<User>> {
        return Observable.concatArray(
            getUsersFromDb(),
            getUsersFromApi())
    }
    fun getUsersFromDb(): Observ-
able<List<User>> {
        return userDao .getUsers().filter { it.is-
NotEmpty() }
            .toObservable()
            .doOnNext {
                Log.d( "Dispatching ${ it. size } users
from DB..." )
            }
    }
}
```

```

fun getUsersFromApi(): Observable<List<User>> {
    return userApi .getUsers()
        .doOnNext {
            Log.d( "Dispatching ${ it. size } users from API..." )
            storeUsersInDb(it)
        }
}

fun storeUsersInDb(users: List<User>) {
    Observable.fromCallable { userDao .insertAll(users) }
        .subscribeOn(Schedulers.io())
        .observeOn(Schedulers.io())
        .subscribe {
            Log.d( "Inserted ${ users. size } users from API in DB..." )
        }
}

```

Here we clearly see the advantage of the Repository Pattern. This repository returns list of users from local db or network. It also have a method to persist users.

These methods can increment against your needs. There is no limit here. But we should design our repository so clean that it should be very easy to use.

As you can see, it's compatible to be used with RxAndroid. Reactive programming in repository is a latest trend in Android world.

Yet another benefit is, repository takes API and Dao as a parameter and work with them. That makes our app more testable.

84. Do you use any ORM libraries in your apps?

Before REALM, there wasn't any successful ORM libraries in Android. I have used this library for a while in my projects. It's first advantage is it is very easy to use as it should be. But there were some problems also with it. You have to extend RealmObject class in your POJO's. That is a disadvantage for inheritance. It also increases APK size tremendously in some cases.

Later then, ROOM has reached out. As a part of Android Architecture Components, ROOM is the only library that i use for persistence in my apps.

It provides compile-time validation for SQL. It's unbelievable! SQL is very open to typos and ROOM saves us.

Secondly it is very easy to use. All you need is a DAO interface. You don't even have to implement the methods. That means no more boilerplate code in database methods.

For instance;

```
@Dao
interface UserDao {
    @Query( "SELECT * FROM user" )
    List<User> getAll();
    @Query( "SELECT * FROM user WHERE uid IN (:userIds)" )
    List<User> loadAllByIds(int[] userIds);
    @Query( "SELECT * FROM user WHERE first_
name LIKE :first AND "
        + "last_name LIKE :last LIMIT 1" )
    User findByName(String first, String last);
```



```
@Insert
void insertAll(User... users);
@Delete
void delete(User user);
}
```

AppDatabase class:

```
@Database( entities = { User. class }, version = 1 )
abstract class AppDatabase: RoomDatabase {
    public abstract UserDao userDao();
}
```

After creating the files above, you get an instance of the created database using the following code:

```
var db = Room.databaseBuilder(getApplication-
Context(),
    AppDatabase:: class .java , "database-
name" ).build()
```

85. You are supposed to build a “URL Shortener” app. How do you design your database?

Ok, when designing database tables we should always design according to objects. So the question should always be “What objects we need?” rather than “What tables we need?”.

Let’s begin with designing our model objects which are called as entity.

Let's call our main entity a **Link** . A Link is a mapping between a **shortLink** on our app, and

a **longLink**, where we redirect people when they visit the shortLink.

So, our data class is,

```
data class Link( val shortLink : String, val longLink : String)
```

Here we are benefitting from Data Class syntax of Kotlin. We don't have to annotate it as an entity or define serialized names. They are optional.

Then of course we need a DAO. I always use DAO pattern in my apps. Since ROOM is the standard library for ORM, we are going to use it.

Here is our DAO object;

```
@Dao
interface UrlShortenerDao{
    @Query( "SELECT * FROM Link WHERE short-Link LIKE :shortLink LIMIT 1 " )
    Link get(shortLink: String);
    @Insert
    void insert(link: Link);
    @Delete
    void delete(link: Link);
}
```

We have three methods: get(), insert(), and delete(). Since this is the first version of our app, we have saved update() method for later.

Actually, it seems pretty simple and efficient. ROOM requires us to write a small query of SQL in our get() method but the rest is pretty easy.

Now we need to write a database class.

```
@Database( entities = { Link. class }, version = 1 )
abstract class AppDatabase : RoomDatabase {
    public abstract UrlShortenerDao urlShortenerDao();
}
```

Now, we are ready to to. All we need is to write a Repository class and call regarding DAO methods from that Repository.

You know, Repository is a pattern. It is the single source of truth for all the app data and it has a clean API for UI to communicate with.

I have to also underline that, i would design my app as three different layers: Presentation, Data and Domain. So far, we have designed our Data layer.

This architecture is called as Clean Architecture and i try to use it in all of my apps. It helps us to satisfy Separation of Concerns Pattern and provides a more robust, scalable, testable structure.



Important:

System design questions like this are usually intentionally left open-ended, so you have to ask some questions and make some decisions about exactly what you're building to get on the same page as your interviewer.

Do not hesitate to ask questions to your interviewer and also make your own decisions. This is supposed to be an interactive question.

You should also mention about architecture and three layers we have mentioned. This is a big plus for you.

CHAPTER 11

ANDROID STUDIO

86. What is ADB? Why do we need it and how can we benefit from it?

ADB is Android Debug Bridge. It is a versatile command-line tool that lets you communicate with a device. Using ADB we can install, uninstall and debug apps on real devices. It basically provides access to a Unix shell that we can run a variety of commands.

The most important commands are as follows:

To connect a device over Wi-Fi:

```
$ adb tcpip 5555  
$ adb connect device_ip_address
```

To list of attached devices:

```
$ adb devices
```

To install an APK on device:

```
$ adb -s emulator-5555 install helloWorld.apk
```

To setup port forwarding:

```
$ adb forward tcp:6100 tcp:7100
```

To copy files to/from a device:

```
$ adb forward tcp:6100 tcp:7100
```

87. What are the most used shortcuts of Android Studio you use?

Our job requires to be working on a computer throughout each standard 8-hour workday, then using keyboard shortcuts can save us 8 entire work days every year. This is equal to 3.3% of our total productivity!

My favorite shortcuts are for formatting and navigation. Here are my favorite shortcuts that I use oftenly.

Shift+Shift — Search Everywhere

Ctrl+F — Find Text with in a Single File

Ctrl+Shift+F -Find Text in All Files

Ctrl+N — Navigate to Class

Ctrl+Shift+N — Navigate to a File

Ctrl+E — Recent Files

Ctrl+Shift+BackSpace — Jump to Last Edited Location

Ctrl + W — Extend selection (selects a word->line->method->Class)

Ctrl + / — Comment / uncomment with line comment

Ctrl+Alt+L — Reformat code

Ctrl + Alt + T — Surround with...

During a long workday, we repeat some common tasks over and over again. These shortcuts saves us time and makes us more productive.

**Important:**

You don't have to memorize these shortcuts. But you should keep in mind some shortcuts for yourself. When you are asked this question, you should at least say "I use shortcuts for, running, debugging, running tests, opening classes etc..."

That is true during a long workday, we repeat some common tasks over and over again. These shortcuts saves us time and makes us more productive.

This is going to help you after interview also :)

88. What is Android Profiler ? How can we use it?

The **Android Profiler** provides real time data for our app's CPU, memory, and network activity . We can perform sample-based method tracing to time our code execution, capture heap dumps, view memory allocations, and inspect the details of network-transmitted files.

Before Android Studio 3.0 there was Android Monitor tools. Android Profiler has replaced that tools.

In android Studio, it is located under **View > Tool Windows > Android Profiler** (we can also click Android Profiler in the toolbar).

89. How do you protect your source codes from being decompiled?

Actually there is no way to stop reverse engineering completely. But we use Proguard for obfuscating our code.

An APK can always be decompiled but if it is obfuscated, things are much complicated to understand. Method names and variable names are obfuscated thanks to Proguard.

Here is how we enable Proguard:

```
android {  
  
    buildTypes {  
  
        release {  
  
            minifyEnabled false  
  
            proguardFiles getDefaultProguard-  
File( 'proguard-android.txt' ), 'proguard-  
rules.pro'  
  
        }  
    }  
}
```



```
}  
  
}
```

But in some cases obfuscation may cause some RunTime Exceptions. Our app may throw ClassNotFoundException interestingly. Generally for libraries and third party classes. We need to keep these classes out of obfuscation.

For this we use:

```
-keep public class MyClass
```

90. You are supposed to deliver two different APK's of your app. (For instance a car booking app for drivers and riders) How can you manage it in same project?

Android Studio provides us **Product Flavors** for this purpose. Creating product flavors is similar to creating build types: add them to the `productFlavors` block in your build configuration and include the settings you want. The product flavors support the same properties as `defaultConfig`—this is because `defaultConfig` actually belongs to the **ProductFlavor** class.

For instance we have a car sharing app and we want to deliver two different APK's for riders and drivers. We should create two product flavors in our project: Driver and Rider.

The following code sample creates a flavor dimension named "version" and adds "driver" and "rider" product flavors. These flavors provide their own `applicationIdSuffix` and `versionNameSuffix`:

```
android {  
    ...  
    defaultConfig {...}  
    buildTypes {  
        debug{...}  
        release{...}  
    }  
    // Specifies one flavor dimension.  
    flavorDimensions "version"  
    productFlavors {  
        driver {  
            // Assigns this product flavor to the "version"  
            // flavor dimension.  
            // This property is optional if you are using only  
            // one dimension.  
            dimension "version"  
            applicationIdSuffix ".driver"  
            versionNameSuffix "-driver"  
        }  
        full {  
            dimension "version"  
            applicationIdSuffix ".rider"  
            versionNameSuffix "-rider"  
        }  
    }  
}
```

91. How do you analyze your apk? Why its size is larger than expected?

Android Studio includes an **APK Analyzer** that provides immediate insight into the composition of our APK after the build process completes.

I generally use this tool to reduce the time i spend debugging issues with DEX files and resources within my app, and help reduce my APK size. It's also available from the command line.

It provides us a bunch of functionalities such as:

- View the absolute and relative size of files in the APK, such as the DEX and Android resource files.

- Understand the composition of DEX files.

- Quickly view the final versions of files in the APK, such as the AndroidManifest.xml file.

- Perform a side-by-side comparison of two APKs.

I see how much space /res folders occupy. If there is a problem with size of resources we should use `shrinkResources` command to delete unused resources.

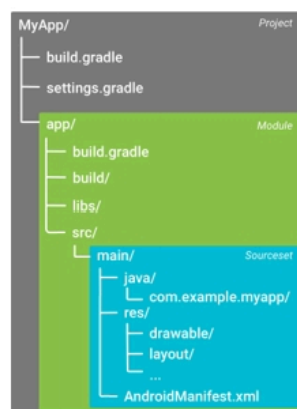
We may also use WebP file format instead of PNG. There are a tons of ways that Google recommends to reduce APK size. These are just a few of them.

92. What do you know about Gradle? How does it work?

Gradle is an advanced build toolkit to automate and manage build process. It allows us to define custom build configurations. Android Studio uses Gradle for build automation. We define dependencies of our project and configure details of our build process.

It was first developed by Gradleware Inc and open-sourced then.

Gradle and the Android plugin run independent of Android Studio. This means that we can build your Android apps from within Android Studio, the command line on our machine, or even on machines where Android Studio is not installed such as continuous integration servers .



In an Android project we have build.gradle file for entire project and a **build.gradle** file for every module.

The `settings.gradle` file, located in the root project directory, tells Gradle which modules it should include when building our app.

The top-level `build.gradle` file, located in the root project directory, defines build configurations that apply to all modules in our project.

To add a dependency to our project, we specify a dependency configuration such as implementation in the dependencies block of our `build.gradle` file.

We are also allowed to add extra properties to the ext block in the top-level `build.gradle` file. So that we can define version numbers as a property and use it for different dependencies that use same version. Such as support libraries.

Creating custom build configurations requires us to make changes to one or more build configuration files, or build.gradle files. These plain text files use **Domain Specific Language (DSL)** to describe and manipulate the build logic using Groovy, which is a dynamic language for the Java Virtual Machine (JVM). But we don't have to know Groovy to build using Gradle. It's straightforward most of the time.

A simple Gradle file should be as follows:

```
apply plugin: 'com.android.application'

android { ... }

dependencies {

    // Dependency on a local library module
```

```
implementation project( ":mylibrary" )

// Dependency on local binaries

implementation fileTree(dir: 'libs' , include:
[ '*.jar' ])

// Dependency on a remote binary

implementation 'com.example.android:app-
magic:12.3'

}
```

CHAPTER 12

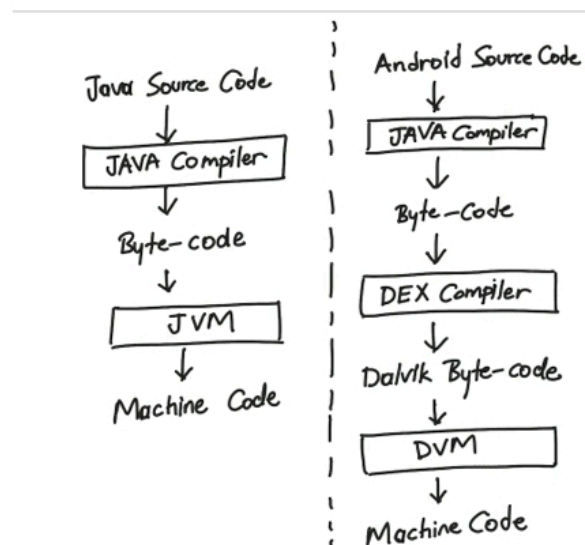
ANDROID SYSTEM INTERNAL

93. How Android code is compiled?

Java code is compiled to an intermediate format called **byte-code**. Then this byte-code is parsed by JVM and translated to machine code.

Android SDK has a tool called **dx** that converts Java byte-code to **Dalvik byte-code**. It is located under `build-tools/android-{version}`.

Then Dalvik VM moves in and translates Dalvik byte-code to binary machine code.



Maybe the most significant difference between the both is that JVM is stack-based while DVM is register-based.

What is stack based and register based?

Ok, let me briefly explain:

In **Stack Based VM's**, all the arithmetic and logic operations are carried out via Push and Pop operands and results in the **stack**.

On the other hand, in **Register Based VM's** , the data structure where the operands are stored is based on the **registers of CPU** .



Important:

Main point behind this question is that your interviewer wants to measure your Computer Science background.

You should definitely know how Android code is compiled. I have provided you a diagram of it. Try to draw it on a paper 10 times. Then when you are asked, take a pen and draw it on a whiteboard or at least on a paper.

Describe the process with drawing. That is the main point of a successful answer in such questions. They want to see if you are able to document solutions with diagrams and teach others easily.

Albert Einstein says that "If you can't explain it simply, you don't understand it enough."

94. What is DEX code?

Dex files are Dalvik executable files for both ART and Dalvik runtimes. They combine the power of our four protagonists creating the bytecode that runs on Android devices.

The dex file is most known for the infamous 65k method count limit but it contains more than just a method table. More precisely, it aggregates content from the app module and all of its dependencies. The dex file format contains the following elements:

1. File Header
2. String Table
3. Class List
4. Field Table
5. Method Table
6. Class Definition Table
7. Field List
8. Method List
9. Code Header
10. Local Variable List

But the road to the dex file is long and winding. First, the Java source code is compiled using `javac` (part of the JDK) to create the `.class` files. Afterwards, using the `dx` tool (part of the Android SDK build tools), the Java bytecode `.class` files are translated to the `.dex` files.



Important:

For more details about DEX code, please refer to “Question 93. How Android code is compiled?”

95. What is ART?

Android runtime (ART) is the managed runtime used by applications and some system services on Android. ART and its predecessor Dalvik were originally created specifically for the Android project. ART as the runtime executes the Dalvik Executable format and Dex bytecode specification.

ART and Dalvik are compatible runtimes running Dex bytecode, so apps developed for Dalvik should work when running with ART. However,

some techniques that work on Dalvik do not work on ART.

Here are some of the major features implemented by ART.

- Ahead-of-time (AOT) compilation
- Improved garbage collection
- Support for sampling profiler
- Support for more debugging features
- Improved diagnostic detail in exceptions and crash reports

ART also provides improved context information in app native crash reports, by including both Java and native stack information.

96. How can you call Garbage Collector in your app?

Nope, you can't call Garbage Collector (GC). Simply, Java doesn't allow us to do this.

In fact, you shouldn't be trying to force GC - if you are running low on memory then you have a memory leak somewhere. Forcing GC at that point won't help, because if you are holding a reference to the object then it still won't be garbage collected.

CHAPTER 13

ADVANCED KOTLIN

97. What is difference between “==” and “===” in Kotlin?

In Kotlin there are two types of equality:

- Structural equality
- Referential equality

Structural equality is checked by the `==` operation. By convention, an expression like `a == b` is translated to:

```
a?.equals(b) ?: (b === null)
```

And its negated counterpart `!=`.

For **referential equality**, we use the `===` symbol which allows us to evaluate the reference of an object if it's pointing to the same object. This is an equivalent of “==” operator in Java.

Let's say we have two integers defined:

```
val a = Integer( 10 )  
val b = Integer( 10 )
```

and we check them both by doing `a === b`, which will return false because they're two

separate objects, each pointing to a different location in memory.

98. How can we define a Singleton object in Kotlin?

In Java, we make constructor private, write a `static getInstance()` method that returns a `private static final` instance of that class.

But in Kotlin, things are much simpler with a much better syntax.

In Kotlin the best way to define a **Singleton** is to create an object . Here it is:

```
object Singleton {  
    init {  
        println ( "This ( $this ) is a singleton" )  
    }  
    var b :String? = null  
}
```

This object is a Singleton. Whenever we instantiate it, we always have the same object over and over again.

```
var first = Singleton //This is a singleton  
first.b = "hello singleton"  
var second = Singleton  
println(second.b) //Prints "hello singleton"  
again.
```



Important:

Singleton is a commonly used Design Pattern, also easy to implement and explain. But keep in mind that singleton is hard to test.

99. What are “Inline Functions” in Kotlin?

inline annotation means that function as well as function parameters will be expanded at call site that means it helps reduce call overhead.

Let me explain it more clearly. High order functions are stored as objects may make the use disadvantageous at times because of the memory overhead. **Inline functions** are introduced for this purpose.

Declaring a function inline is simple, just add inline keyword to the function like in below code snippet :

```
inline fun someMethod(a: Int, func: () -> Unit):Int
{
    func()
    return 2 *a
}
```

100. How to declare mutable and immutable objects in Kotlin?

In Kotlin variables are declared using `var` or `val`, provided they are **mutable** or **immutable**.

`var` means this object is mutable, it can be re-assigned, its value can be changed.

On the other hand, `val` means this object is immutable. Its value can not be changed. It's like `final` keyword in Java.

An example of mutable objects:

```
var x = 7
var y : String = "my String"
var z = View( this )
```

But in Kotlin **immutable** objects are always preferred.

```
val x : Int = 20
val y : Double = 21.5
val z : Unit = Unit
```

Due to convention of using `val` whenever possible, Kotlin provides much more safety.

101. What is a “Higher Order Function” in Kotlin?

A **higher-order function** is a function that takes functions as parameters, or returns a function.

Kotlin functions are *first-class*, which means that they can be stored in variables and data structures.

For instance here we have a `higherOrderFunction()` that takes another function as a parameter. The thing is we need to define input and output

parameters of incoming function. Then inside `higherOrderFunction()` we can use this `fn` function.

```
fun higherOrderFunction(fn: () -> Unit){  
    fn()  
}
```

How can we benefit this?

We can change behavior of methods at runtime. For instance, we can send different methods with different algorithms to be executed at a higher order function.

Here we have two simple methods: `add` and `subtract`. Both of them takes two `Integers` and return an `Integer` as a result. Their parameter signature is same.

```
val add = fun (num1: Int, num2: Int): Int{  
    return num1 + num2  
}  
val subtract = fun (num1: Int, num2: Int): Int{  
    return num1 - num2  
}
```

In Kotlin, we can assign functions to variables.

Then we can write a higher order function to take one of this functions with two integers. Then it's going to execute incoming function with incoming parameters.

Here our higher order function:

```
fun higherOrderFunction(fn: (param1: Int,  
param2: Int) -> Int, num1: Int, num2: Int){  
    val result = fn(num1, num2)  
    print ( "Result is $ result " )  
}
```

As you can see, it defined parameter signature of incoming function: takes two `Integers` and return an `Integer`.

```
fn: (param1: Int, param2: Int) -> Int
```

Then we can call this method using both of the methods.

```
higherOrderFunction( add , 15 , 10 ) // prints 25  
higherOrderFunction( subtract , 13 , 8 ) // prints  
5
```

This how can we benefit from higher order functions.

102. What are “open”, “final” and “abstract” modifiers in Kotlin?

open : In Kotlin, classes are closed to be extended. That means by default, all classes are **final**. We have to use **open** keyword to make classes extendable.

open keyword can also be used with functions and properties. A function or property with **open** keyword can be overridden. Otherwise they are not allowed to be overridden.

Extending an open class:

```
open class Base(p: Int)  
class Derived(p: Int) : Base(p)
```

Overriding a function:

```
open class Base {  
    open fun v() { ... }  
}  
class Derived () : Base() {  
    override fun v() { ... }  
}
```

Overriding a property:

```
open class Foo {  
    open val x : Int get () { ... }  
}  
class Bar1 : Foo() {  
    override val x : Int = ...  
}
```

By default, all classes in Kotlin are final, which corresponds to **Effective Java** book Item 19: “*Design and document for inheritance or else prohibit it.*”.

final: Forbids overriding a member.

We have said classes and members are already closed to be extended. But why we need it?

Answer lies here: when a function or a property is overridden, it is open to inheritance. We use final keyword to close it to be overridden again.

For instance:

```
open class Base {  
    open fun v() { ... }  
}  
open class Derived () : Base() {  
    final override fun v() { }  
}  
class DerivedSub () : Derived() {  
    override fun v() { ... } // function v() is final and  
    cannot be overridden  
}
```

Base class is **open** and it has `v()`. Since it is defined **open**, it can be overridden.

Derived class is also **open** and it extends **Base**.

Derived has also overridden **open fun v()**.

override keyword in **override fun v()** makes this method open to inheritance. But when this method is **final override**, then it's close to inheritance.

`DerivedSub` class tries to **override fun** `v()` but compiler gives error: “*function v() is final and cannot be overridden*”

To sum up, a member with **final** keyword cannot be overridden.

Abstract: An **abstract** class is a class that cannot be instantiated. We create abstract classes to provide a common template for other classes to extend and use.

Functions and properties can also be declared **abstract**. Those members have to be overridden by concrete subclasses.

An **abstract** class is also **open**. We don't need to use **open** keyword, it is **open** by default.

```
abstract class Device( val name : String,
                       val color : String,
                       val weight : Double) { // Concrete (Non
Abstract) Properties
    // Abstract Property (Must be overridden by
Subclasses)
    abstract var maxSpeed : Double
    // Abstract Methods (Must be implemented by
Subclasses)
    abstract fun start()
    abstract fun stop()
    // Concrete (Non Abstract) Method
    fun displayDetails() {
        println ( "Name: $ name , Color: $ color ,
Weight: $ weight , Max Speed: $ maxSpeed " )
    }
}
```

One important detail is and **abstract** class may have **abstract** and non-abstract methods. And we can override a non-abstract open member with an **abstract** one:

```
open class Base {
    open fun f() {}
}
```

```
abstract class Derived : Base() {  
    override abstract fun f()  
}
```



Red Flag:

Interviewer wants to see if you are comfortable with **Object Oriented Programming** concepts.

Don't you even dare to mix meanings of **open**, **final** and **abstract**. These are basic keywords for **Object Oriented Programming**. We use these keywords for implementing **Encapsulation**, **Inheritance** and **Polymorphism**. Actually, Encapsulation is more related with access modifiers but these keywords are very important for other two concepts: Inheritance and Polymorphism.

Interviewer may also ask you these three concepts, you should be prepared.

For more detailed information i can provide you a nice article. Please refer to <https://medium.freecodecamp.org/object-oriented-programming-concepts-21bb035f7260>

103. What are Data Classes in Kotlin? What makes them so useful?

Data Classes are classes that holds data. Their only purpose is this. Pure, simple and elegant. We use them for model classes in our projects.

```
data class Book( val name : String, val publisher :  
String, var reviewScore : Int)
```

Data classes use primary constructor and there must be at least one primary constructor parameter.

Parameters can be **val** or **var**. They may also have visibility modifiers such as **private**.

We instantiate a Data class as a normal object:

```
val book = Book( "1984" , "George Orwell" , 10 )
```

Kotlin also provides us a `copy()` function to easily copy data classes.

```
val book = Book( "1984" , "George Orwell" , 10 )  
val otherbook = book .copy( name = "Animal  
Farm" )
```

Here we see, `copy()` method has copied book object and we have changed one of its properties. The rest is the same but it's a new object with a new place on memory heap.

104. What is “Pair” and “Triple” in Kotlin?

Kotlin gives us **Pair** and **Triple** classes for holding more than one data in an object. Using Pair or Triple objects, our functions can return two or three values at the same time.

That saves us from writing a new class for data pairs. Let's assume we have a method that returns a user's name and age. How can you do it?

We definitely write a User class which has name and age fields and our function returns a User object.

But thanks to Kotlin, we no longer have to define a new class. Because we have **Pair** and **Triple** classes.

Here how to use it:

```
val user = Pair( "John" , 25 )  
println (user. first ) // prints John  
println (user. second ) // prints 25
```

Kotlin by default names pair items as **first** and **second**. We have to use this syntax to reach values.

We are also allowed to define Pair objects. User object created above can be used as:

```
val (name, age) = user  
println (name) // prints John  
println (age) // prints 25
```

Triple is same as Pair. The only difference is Pair can hold two values while Triple can three.

An example of Triple would be:

```
val actor = Triple( "John" , "Wick" , 25 )  
println (actor. first ) // prints John  
println (actor. second ) // prints Wick  
println (actor. third ) // prints 25
```

To sum up, if we want to hold more than one data in an object without creating a special class for it, Kotlin provides us **Pair** and **Triple** classes.

105. What is difference between “lazy” and “lateinit” ?

Using **lateinit**, the initial value does not need to be assigned. However, we have to be careful to assign our lateinit var a value before we use it. Otherwise, a lateinit property acts as if we performed `!!`: it will crash the app on a null value.

A property defined via by **lazy** is initialized using the supplied lambda upon first use, unless a value had been previously assigned.

Lazy is a good fit for properties that may or may not be accessed. If we never access them, we avoid computing their initial value. They may work for Activities, as long as they are not accessed before `setContentView` is called. They are not a great fit for referencing views in a Fragment, because the common pattern of configuring views inside `onCreateView` would cause a crash. They can be used if view configuration is done in `onViewCreated`.

With Activities or Fragments, it makes more sense to use **lateinit** for their properties, especially the ones referencing views. While we don't control the lifecycle, we know when those properties will be properly initialized. The downside is we have to ensure they are initialized in the appropriate lifecycle methods.

CHAPTER 14

COMMON MISTAKES & ERRORS

106. How can you reduce APK size?

We should remove unused resources . There is a tool in Android Studio called **lint** for this purpose. The **lint** tool detects resources in our `res/` folder that our code doesn't reference.

Libraries that we use may include unused resources . We may eliminate them with **shrinkResources** in our app's `build.gradle` file.

```
android {  
    // Other settings  
    buildTypes {
```

```
release {
    minifyEnabled true
    shrinkResources true
    proguardFiles getDefaultProguard-
File( 'proguard-android.txt' ), 'proguard-
rules.pro'
}
}
```

We may support only specific densities . If we know that only a small percentage of your users have devices with specific densities, we may bundle those densities into our app. Android automatically scales existing resources originally designed for other screen densities.

We may prefer XML drawables or WEBP file format over PNG or JPG ones. The WebP format provides lossy compression as well as transparency but can provide better compression than either JPEG or PNG.

We can use vector graphics to create resolution-independent icons and other scalable media. Using these graphics can greatly reduce our APK footprint.

We should use **AnimatedVectorDrawableCompat** to create animated vector drawables instead of **AnimationDrawable** . AnimationDrawable requires that we include a separate bitmap file for each frame of the animation, which drastically increase the size of our APK.

On the code side, we may remove unnecessary generated code or avoid enumerations . A single enum can add about 1.0 to 1.4 KB of size to our app's `classes.dex` file. These additions can quickly accumulate for complex systems or shared libraries.

107. Do you know what is inside of an APK file?

An APK contains the following directories:

META-INF/: Contains the CERT.SF and CERT.RSA signature files, as well as the MANIFEST.MF manifest file.

assets/: Contains the app's assets, which the app can retrieve using an `AssetManager` object.

res/: Contains resources that aren't compiled into `resources.arsc`.

lib/: Contains the compiled code that is specific to the software layer of a processor. This directory contains a subdirectory for each platform type, like `armeabi`, `armeabi-v7a`, `arm64-v8a`, `x86`, `x86_64`, and `mips`.

An APK also contains the following files. Among them, only *AndroidManifest.xml* is mandatory.

resources.arsc: Contains compiled resources. This file contains the XML content from all configurations of the `res/values/` folder. The packaging tool extracts this XML content, compiles it to binary form, and archives the content. This content includes language strings and styles, as well as paths to content that is not included directly in the `resources.arsc` file, such as layout files and images.

classes.dex: Contains the classes compiled in the DEX file format understood by the Dalvik/ART virtual machine.

AndroidManifest.xml: Contains the core Android manifest file. This file lists the name, version, access rights, and referenced library files of the app. The file uses Android's binary XML format.

108. Your app is not looking good on tablets. How can you solve this problem?

First of all we need to create a flexible layout .

The best way to create a responsive layout for different screen sizes is to use **ConstraintLayout** as the base layout in our UI.

ConstraintLayout allows us to specify the position and size for each view according to spatial relationships with other views in the layout. This way, all the views can move and stretch together as the screen size changes.

We should create stretchable nine-patch bit-maps. This is a special kind of images that works fine on Android.

We should avoid hard-coded layout sizes. Instead we should use `wrap_content` or `match_parent` .

We should create alternative layouts . The app should provide alternative layouts to optimise the UI design for certain screen sizes. For eg: different UI for tablets.

We should use the smallest width qualifier. For instance, we can create a layout named `main_activity.xml` that's optimised for handsets and

tablets by creating different versions of the file in directories as follows:

`res/layout/main_activity.xml` — For handsets (smaller than 600dp available width)

`res/layout-sw600dp/main_activity.xml` — For 7" tablets (600dp wide and bigger).

The smallest width qualifier specifies the smallest of the screen's two sides, regardless of the device's current orientation, so it's a simple way to specify the overall screen size available for our layout.

We should test text and graphics on *ldpi* and *mdpi* screen densities to ensure that our text and graphics work well on low- and medium-density screens.



Important:

Yet another generic question. As an Android developer, your daily life will be dominated with effort of making sure your app is looking good on different devices. You are developing for phones and tablets. Interviewer wants to see that you are capable of handling UI issues with tablets. Most of the times, apps are responsive and it may take a bit time to ensure this responsiveness. Your employer wants to be sure that you are good enough to handle that.

Just repeat the given answer. That will be well enough.

109. How can you optimize your app's battery consumption?

There are several steps we can take to help make sure that our app is only consuming battery power when it needs to, and that it's not consuming more power than necessary.

Our app should minimize its activity when in the background and when the device is running on battery power.

Sensors, such as GPS sensors, can drain battery significantly. We should avoid issues by using the **FusedLocationProvider API** to manage the underlying location technology. It provides a simple API so that you can specify requirements such as high accuracy or low power at a high level. It also optimizes the device's use of battery power by caching locations and batching requests across apps.

We should avoid using wake locks because they prevent the device from going into low-power states.

We may batch network activity to reduce the number of device wake-ups.

We should use **GcmNetworkManager** to perform non-essential background activity when the device is charging and is connected to an unmetered network.

GcmNetworkManager schedules tasks and lets Google Play services batch operations across the system. This greatly simplifies the implementation of common patterns, such as waiting for network connectivity, device charging state, retries, and backoff.

110. How do you protect your API keys in your project?

So that no one sees them in committed code on repo.

We should never hard code keys in code. We should always add the key to our `gradle.properties` file in our home directory under `.gradle` directory.

Then we may import the key as a `buildConfigField` / `resValue` in our module-level `build.gradle` file.

Then we are ready to use the key in our code or XML files as we need.

For instance, `build.gradle` file:

```
buildTypes {  
    debug {  
        buildConfigField 'String', "ApiKey",  
        My_ApiKey  
        resValue 'string', "api_key", My_ApiKey  
    }  
  
    release {  
        buildConfigField 'String', "ApiKey",  
        My_ApiKey  
        resValue 'string', "api_key", My_ApiKey  
    }  
}
```

Then in Kotlin code, we may get the key as follows:

```
val apiKey: String = BuildConfig.ApiKey;
```